

On the Integration of Spectrum-Based Fault Localization Tools into IDEs

Attila Szatmári
Department of Software Engineering,
University of Szeged
Szeged, Hungary
szatma@inf.u-szeged.hu

Qusay Idrees Sarhan
Department of Computer Science,
University of Duhok
Duhok, Iraq
qusay.sarhan@uod.ac

Gergő Balogh
Department of Software Engineering,
University of Szeged
Szeged, Hungary
geryxyz@inf.u-szeged.hu

Péter Attila Soha
Department of Software Engineering,
University of Szeged
Szeged, Hungary
psoha@inf.u-szeged.hu

Árpád Beszédes
Department of Software Engineering,
University of Szeged
Szeged, Hungary
beszedes@inf.u-szeged.hu

ABSTRACT

Spectrum-Based Fault Localization (SBFL) is a technique to be used during debugging, the premise of which is that, based on the test case outcomes and code coverage, faulty code elements can be automatically detected. SBFL is popular among researchers because it is lightweight and easy to implement, and there is a lot of potential in it when it comes to research that aims to improve its effectiveness. Despite this, the technique cannot be found in contemporary development and debugging tools, only a handful of research prototypes are available. Reasons for this can be multiple, including the algorithms' sub-optimal effectiveness and other technical weaknesses. But, also the lack of clear functional and non-functional requirements for such a tool, either standalone or integrated into IDEs. In this paper, we attempt to provide such a list in form of recommendations, based on surveying the most popular SBFL tools and on our own researchers' and tool builders' experience.

CCS CONCEPTS

• **Software and its engineering** → **Integrated and visual development environments; Software testing and debugging; Empirical software validation.**

KEYWORDS

Spectrum-Based Fault Localization, SBFL, IDE, debugging.

ACM Reference Format:

Attila Szatmári, Qusay Idrees Sarhan, Gergő Balogh, Péter Attila Soha, and Árpád Beszédes. 2024. On the Integration of Spectrum-Based Fault Localization Tools into IDEs. In *2024 First IDE Workshop (IDE '24)*, April 20, 2024, Lisbon, Portugal. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/3643796.3648448>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

IDE '24, April 20, 2024, Lisbon, Portugal

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-0580-9/24/04...\$15.00

<https://doi.org/10.1145/3643796.3648448>

1 INTRODUCTION

Spectrum-Based Fault Localization (SBFL) is a lightweight debugging technique that profiles the program's execution through its test cases, collects code coverage and test results (collectively called the spectrum), and then, based on statistics of failing and passing test cases and their relation to code elements, assigns suspiciousness scores to each such element, a statement or function in particular [4, 9, 30]. We provide a brief overview of the technique in Section 2.

Although it is a well-researched topic, SBFL is not widely adopted in the industry, and the research community is mostly aware of reasons why SBFL can be challenging. Sometimes meeting technical requirements can be difficult, and due to its statistical nature, SBFL can occasionally mislead developers during the debugging process leading to the rejection of the technique by practitioners [17, 19]. Sarhan and Beszédes identified additional challenges in SBFL [24].

In this paper, we concentrate on the aspects of implementing SBFL techniques into practical tools, in particular their integration into Integrated Development Environments (IDEs). Hence, we do not deal with any other technical or theoretical aspects of the technique. With this, we distinguish our study from a more conceptual comparison of related tools such as the one performed by Archana and Agarwal [6].

We base our investigation on three sources. First, we perform a lightweight survey of existing SBFL tools to identify their main properties, and more importantly, the existing strengths and weaknesses (Section 3). Second, we summarize our findings from a recent empirical study that we performed with actual users who have been given a debugging task to be performed in an IDE with some of our own SBFL tools (Section 4). Third, we collect our own experience as researchers working in the SBFL domain for several years and as tool builders who have experience in integrating debugging tools into various IDEs including Eclipse and JetBrains.

We list our findings in Section 5 from three perspectives: users', developers, and researchers', with the hope that future tool builders in this area could benefit from it when designing new tools, preferably integrated into IDEs.

$$\begin{aligned}
 & \begin{matrix} n \text{ source code items} \\ C = \begin{bmatrix} c_{1,1} & c_{1,2} & \cdots & c_{1,n} \\ c_{2,1} & c_{2,2} & \cdots & c_{2,n} \\ \vdots & \vdots & \ddots & \vdots \\ c_{m,1} & c_{m,2} & \cdots & c_{m,n} \end{bmatrix} \end{matrix} \quad \begin{matrix} m \text{ tests} \\ e = \begin{bmatrix} e_1 \\ e_2 \\ \vdots \\ e_m \end{bmatrix} \end{matrix} \\
 \\
 & \begin{matrix} \text{metric}_{\text{exec,res}}(j) = \left| \{i \mid M_{i,j} = \text{exec} \wedge e_i = \text{res}\} \right| \end{matrix} \quad \begin{matrix} 4 \text{ basic metrics} \\ \text{exec, res} \in \{0, 1\} \end{matrix} \\
 \\
 & s = \begin{bmatrix} s_1 & s_2 & \cdots & s_n \end{bmatrix} \quad \begin{matrix} n \text{ score values} \end{matrix}
 \end{aligned}$$

Figure 1: Overview of SBFL

2 SPECTRUM-BASED FAULT LOCALIZATION

SBFL is a lightweight statistical fault localization approach in which every element (such as a statement or a function) will be assigned a suspiciousness score (usually between 0 and 1), indicating the possibility that the fault is caused by that element [2]. This score is then used to provide a ranked list of potentially faulty code elements to the user who can rely on this information during debugging.

To calculate these scores, SBFL operates on the so-called program spectrum. It consists of a coverage matrix and a test result vector. In the most common case, the rows of the coverage matrix denote test cases of the program and the columns represent the elements of the source code. A cell value, $c_{i,j}$ is 1 if during the execution of the i^{th} test case the j^{th} element was executed, and 0 otherwise. The test result vector (e) represents the results of the executions of the test cases, where e_i is equal to 1 if the i^{th} test case failed and 0 otherwise (Figure 1).

The next step is to calculate suspiciousness scores, which is usually done based on four basic metrics, where $\text{metric}_{\text{exec,res}}(j)$ is the number of runs when the j^{th} element was executed ($\text{exec} = 1$) or not ($\text{exec} = 0$) and whether the inspected test case failed ($\text{res} = 1$) or passed ($\text{res} = 0$). These four basic metrics are used to compose various suspiciousness formulas, such as Tarantula [16], Ochiai [1], or Barinel [3].

Researchers advised many different types of extensions to this basic approach, including program slicing [29], adding contextual information [10], user feedback [13], and machine learning [12], to enhance the effectiveness.

3 SBFL TOOLS

In this section, we provide an overview of the most frequently cited SBFL tools, highlighting their main features. It is worth noting that our research does not take into account non-functional requirements like speed or memory usage. Nevertheless, we acknowledge that these factors are crucial for boosting the acceptance of SBFL tools in the industrial sector, and we plan to address them in follow-up research.

Due to the nature of SBFL, it calls for its implementation into an IDE directly rather than as a standalone tool, but there are examples for both.

Wang *et al.* [28] proposed a fault localization tool named “FLAVS” for developers utilizing the Microsoft Visual Studio platform. The tool collects program spectrum information during program execution to identify suspicious program elements. It monitors all environmental parameters of the running program, such as thread counts, CPU use, and memory usage. Chen and Wang [8] have expanded the features of “FLAVS” to another tool named “UnitFL”. To shorten the program execution time, the tool makes use of program slicing.

A tool called “SFLaaS” was proposed by Sarhan *et al.* [23] to locate faults in Python programs. It is offered as a cloud-based service instead of a plugin or command-line tool. The program offers various tie-breaking techniques, approximately 80 SBFL formulas, the ability for the users to define their own formulas, and the display of code elements in various colors according to their suspiciousness scores, from most suspicious (red) to not suspicious (green).

For Java developers, Ribeiro *et al.* [22] proposed the SBFL tool “Jaguar”. The tool supports control and data flow, the two advanced spectra types. Moreover, it presents suspicious program elements graphically so that the user can quickly examine suspicious variables, statements, or methods.

“Whyline” is a debugging tool for Java developers that was proposed by Ko and Myers [18]. The tool utilizes slicing to aid developers in understanding the program behavior under test through a set of questions that are presented in a graphical and interactive way. The tool also logs program execution traces and the status of each program element, indicating its execution status. The tool allows users to view a program’s execution trace and select an element at a specific execution point for examination.

The Eclipse plug-in tool “VIDA” was proposed by Hao *et al.* [14] for Java programs. The tool provides the top ten most suspicious elements as possible breakpoints, along with a history of previous breakpoints and their current level of suspicion. Static dependency graphs are also produced to assist developers in comprehending the relationships between program elements.

A tool called “GZoltar” was proposed by Campos *et al.* [7] to locate faults in Java programs, and it is provided as a plugin for the Eclipse IDE. It generates visually appealing and interactive diagnostic report visualizations like Sunburst and Treemap.

A tool called “CharmFL” was proposed by Sarhan *et al.* [25] to locate faults in Python programs using SBFL. The tool can be used as a standalone or as a plugin for the PyCharm IDE. It supports different code coverage types (i.e., statements, methods, and classes) with the possibility to investigate these types in a hierarchical approach.

The “CharmFL” tool has been extended in [27] to support interactivity in fault localization. With the help of this feature, users can provide their feedback on the suspicious elements and help reorder them, which may improve the fault localization process. This is achieved by utilizing contextual information (i.e., function call graphs) during a failed test. As a result, users are able to examine the data flow traces of any failed test and pinpoint the fault’s location more precisely. This interactivity feature is also provided by “iFL4Eclipse” an Eclipse plug-in SBFL tool [15].

4 EXPERIENCES FROM A USER STUDY

In a recent study, empirical experiments were conducted to assess the effectiveness of the SBFL technique with user feedback integrated into an IDE [15]. The research was performed on different levels; with simulation, using students, and with professional developers. Among other things, conclusions could be drawn from this study regarding the user's perception of the tool used, so this information is very useful for our purposes in the present paper.

We reuse this data from the user study, in particular, the participants' oral responses in form of transcripts of the experiment sessions. For constructing the transcripts, a *think-aloud technique* was utilized which was performed with six professional developers. It involved 24 debugging sessions, with each of the developers participating in four of them. Their task was to find and fix a bug in existing software using an SBFL tool plug-in in the Eclipse IDE. During the process, their ways of thinking were recorded, which included navigating and understanding the code, and in particular using the tool's functions.

In the original paper by Horváth *et al.* [15] the main goal was to evaluate the effectiveness of interactive FL in a variety of scenarios. The tasks for the experiments were selected from various projects by choosing bugs that did not require specific domain knowledge. This approach was intended to simulate situations where developers debug an unfamiliar program, which is common in real-world software development. The participant pool included undergraduate and graduate students as well as professional developers, aiming to cover a diverse range of experience levels. The experimental setup was balanced in terms of participant distribution across different groups. The complexity of tasks and potential prior knowledge of the participants about projects or bugs was considered during the design phase.

We processed the transcript of these sessions with a focus on the participants' opinions about the tools. To enhance objectivity, we instructed the ChatGPT 4.0 LLM engine [21] to extract all the expectations (both implicit and explicit) of the software engineers from the transcripts. Implicit expectations refer to improvements in the IDE and plug-ins that make their tasks easier, which can be inferred from their actions and behavior. On the other hand, explicit expectations are specific requests for features or explicitly expressed wishes by the developers.

The recommendation extraction process is shown in Figure 2. The transcripts of the think-aloud sessions, prompt instructions and metadata from the previous experiment are entered into ChatGPT to help process and summarize these diverse data sources. All prompts are available in the online appendix of this paper [26]. Users' proposals for integration to the IDE and their opinions on the current level of integration summarized by ChatGPT were then carefully investigated by the authors of this paper. We treat this step as an expert evaluation as we could contribute valuable insights based on our experience with the development of SBFL tools. We then constructed a list of proposals and opinions. Depending on their actionability, recommendations for integration with the IDE are formulated or questions are raised that address open challenges and issues.

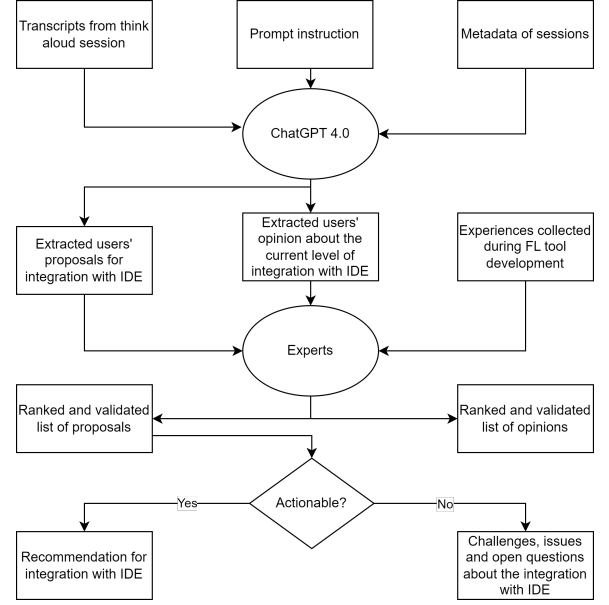


Figure 2: Recommendation extraction process

ChatGPT extracted more than 150 expectations from over 250 thousand tokens' transcripts. We utilized a tool to filter semantic duplicates, which resulted in 17 distinct raw expectations. To validate the raw expectations, we used the Elo rating system [5, 11], typically used for calculating the relative skill levels of players in zero-sum games like chess. In this case, each expectation was considered a "player" competing for the votes of software engineering experts who had to decide which expectation was more important during debugging or fault localization. To determine the winner, we counted the votes for each expectation and updated their Elo rating accordingly. We used two free scaling parameters, K and c, with their typical values of K=32 and c=400. Each expectation had an initial rating of 1500. Our matching strategy combined elements of randomness, exploration, and exploitation to ensure a balanced and fair evaluation of all items. The source code of our rating strategies is available in the online appendix [26].

In the final phase, we compared the expectations that were ranked with the features provided by the fault localization tools. This helped us create the final recommendations and assemble the list of open questions, to be discussed in Section 5.

5 RECOMMENDATIONS

In this section, we summarize and blend our findings from the tool survey in Section 3, the user study from Section 4, and our own experiences. First, we give an overall list of expectations we identified and assess the investigated tools from the perspective of how they implement them in Section 5. In the last part of this section, we summarize what we believe are the most important recommendations from the users', developers', and researchers' perspectives, respectively.

Section 5 is composed of two parts: expectations from the user study in the upper half and other generic expectations based on

Table 1: Summary of expectations and their relationship to existing SBFL tools.

C = CharmFL, F = FLAVS, G = GZoltar, J = Jaguar, S = SFLaaS, U = UnitFL, V = VIDA, W = Whyline, i = iFL4Eclipse; red = no tool implements or plans it, blue = less then 3 tools (1st quartile) implement or plan it; ⦿ = responsibility is unassigned; ♣ = IDE is responsible; ♠ = plug-in is responsible; Responsibilities were determined based on our experiences, taking into account expectations, their manifestations in tools, and tools' visions.

Short Summary	Rating	Implementation		None	Validation			Maturity		Ecosystem			
		Implemented	Planned		by Authors	with Students	with Prof.	Alpha Version	Beta Version	Stand-alone	Eclipse	Visual Studio	PyCharm
code navigation and search	1.00	6 (C,F,G,J,S,U,V,W)		3 (F,G,U)		2 (J,i)	1 (i)	1 (C)	5 (F,G,J,U,i)		3 (G,J,i)	2 (F,U)	1 (C)
few clearer error messages	0.90	1 (G⦿)		1 (G)					1 (G)		1 (G)		
few tests' running optimization	0.88		2 (G⦿,⦿)	2 (G,J)					2 (G,J)		2 (G,J)		
code execution path visualization	0.88	1 (J⦿)		1 (J)					1 (J)		1 (J)		
few clearer debugging process	0.86	2 (V⦿,W⦿)		1 (V)		1 (W)	1 (W)	1 (V)	1 (W)	1 (W)	1 (V)		
intuitive UI	0.86	7 (C,F,G,J,S,U,V,W)		5 (F,G,J,S,U)	1 (C)	2 (W,i)	2 (W,i)	2 (C,S)	5 (F,G,J,U;W)	1 (W)	1 (G,J)	1 (F,U)	1 (C)
few advanced debugging tools	0.83	1 (W⦿)				1 (W)	1 (W)		1 (W)				
responsive IDE	0.81	3 (C⦿,J,i,F,G,⦿)		3 (C,G,J)				1 (C)	2 (F,G)		1 (G)	1 (F)	1 (C)
none customizable interface	0.79	⦿	⦿	⦿	⦿	⦿	⦿	⦿	⦿	⦿	⦿	⦿	⦿
intuitive code coverage visualization	0.78	2 (C⦿,G⦿)	1 (J⦿)	3 (C,G,J)				1 (C)	2 (G,J)		2 (G,J)		1 (C)
streamlined test processes	0.77	8 (C,F,G,J,S,U,V,W;U;V;W;⦿)		7 (C,F,G,J,S,U;V)		1 (i)		3 (C,S;V)	5 (F,G;U;i)	1 (S)	4 (G,J;V;i)	2 (F,U)	1 (C)
IDEs' compatibility with plugins	0.70	(C,F,G,J,S,U)		(C,G,U)				1 (C)	2 (G,U)		2 (G,U)		1 (C)
none code changes' error message clarity	0.69	⦿	⦿	⦿	⦿	⦿	⦿	⦿	⦿	⦿	⦿	⦿	⦿
plugin integration and interaction	0.61	3 (C⦿,G⦿,U⦿)		3 (C,G,U)				1 (C)	2 (G,U)		1 (G)	1 (F)	1 (C)
none integrated domain knowledge	0.60	⦿	⦿	⦿	⦿	⦿	⦿	⦿	⦿	⦿	⦿	⦿	⦿
none integrated documentation	0.00	⦿	⦿	⦿	⦿	⦿	⦿	⦿	⦿	⦿	⦿	⦿	⦿
interactivity between SBFL and developers	-	3 (C,F;V;⦿)	1 (S⦿)	3 (C,S;V)			1 (i)	3 (C,S;V)	1 (i)	1 (S)	2 (V;i)		1 (C)
few hierarchical view based on code structure	-	3 (C⦿,G⦿,⦿)		2 (C,G)		1 (J)		1 (C)	2 (G,J)		2 (G,J)		1 (C)
few accessibility-focused UIs	-	1 (G⦿)		1 (G)				1 (G)			1 (G)		
none enabling teamwork and sharing	-	⦿	⦿	⦿	⦿	⦿	⦿	⦿	⦿	⦿	⦿	⦿	⦿
few inspect suspicious items from different aspects	-	1 (J⦿)				1 (J)			1 (J)		1 (J)		
few optimize SBFL with test selection & prioritization	-	1 (V⦿)		1 (V)				1 (V)			1 (V)		
none incorporate AI into SBFL	-	⦿	⦿	⦿	⦿	⦿	⦿	⦿	⦿	⦿	⦿	⦿	⦿
none handling code element context switching	-	⦿	⦿	⦿	⦿	⦿	⦿	⦿	⦿	⦿	⦿	⦿	⦿
none on-demand reanalysis upon code change	-	⦿	⦿	⦿	⦿	⦿	⦿	⦿	⦿	⦿	⦿	⦿	⦿
suspiciousness score visualization	-	5 (F⦿,G⦿,⦿,⦿,⦿)	1 (S⦿)	5 (F,G;S;U;V)		1 (J)		2 (S;V)	4 (F,G;J;U)	1 (S)	3 (G,J;V)	2 (F,U)	
none understanding the tool's reasoning	-	⦿	⦿	⦿	⦿	⦿	⦿	⦿	⦿	⦿	⦿	⦿	⦿
none (smart) customizable feature set	-	⦿	⦿	⦿	⦿	⦿	⦿	⦿	⦿	⦿	⦿	⦿	⦿
few customizable SBFL algorithm	-	2 (F;S⦿)		2 (F;S)					2 (F;S)	1 (S)		1 (F)	

the tool survey and the authors' own experiences in the lower half. The first column is a short summary of the expectation with the associated Elo rating in the second column (for user study only). Columns 3 and 4 indicate if the expectation is implemented or planned in the respective tools, columns 5–8 show how the feature has been validated, columns 9 and 10 estimate the tool's maturity, and the last 4 columns show how the tool is implemented, either a standalone or an IDE plugin.

Note that since we identified a relatively large number of expectations (column one of Section 5), due to space limits we cannot explain all in detail; their description can be found in the online appendix [26].

5.1 Users perspective

Based on the data presented in Section 5, we identified two interwoven trends in user expectations. First, users expect SBFL tools to be seamlessly integrated in their workflow and efficient. Second, users prefer these tools to be easy to understand and use.

Recommendation 5.1.1: Seamless integration into IDEs and developers workflow. For developers, IDEs are considered complete tools that help them in their daily workflow rather than just collections of plug-ins. They expect the UI to be intuitive, along with various visualizations such as coverage and suspiciousness score. The plug-ins should work cohesively with each other and with the IDE to meet the users' expectations.

Recommendation 5.1.2: Transparent reasoning behind the tool. As problem solvers, developers love to automate complex processes. However, they may not fully trust the SBFL algorithms due to the lack of familiarity. To earn their trust, SBFL tools should

provide clear error messages, easily accessible documentation, explainable operation, and drill-down views that help developers understand the reasoning behind the tool's suggestions.

5.2 Developers perspective

In this section, we provide advice on how to incorporate SBFL tools into IDEs, based on our experience as developers.

Recommendation 5.2.1: Find efficient ways to produce, store and process runtime data. The SBFL spectrum, including test case execution information, should be kept in a data structure that allows for quick access to its individual elements. This is essential if we want to ensure efficient operation with a responsive user interface of the tool.

Recommendation 5.2.2: Programmers should strive to fully utilize IDEs' data and API features. The IDE should take charge of certain components of the SBFL process, and the developer should work together with the IDE to make sure that the process is handled properly and minimize solutions outside of the IDE. For instance, it is recommended that test execution is done directly within the IDE.

Recommendation 5.2.3: Ensure that code elements are easily identifiable. IDEs store program execution data in their own format, which may differ from the static source code structure. SBFL tools require both dynamic analysis and static source code representation, especially when working with loosely typed scripting languages like Python or JavaScript. Information from static and dynamic analysis is often a challenge to match.

Recommendation 5.2.4: Reporting about validation efforts. During the inspection of the selected set of tools against the expectations collected from users and developers, we noticed a concerning trend regarding the authors' validation efforts. Although most of the papers did a good job of reporting the features and methods

underlying their research, in many cases we were unable to find concrete evidence of how the authors validated their contributions, and we believe this should be improved.

Recommendation 5.2.5: Decide on the responsibility of tool and IDE developers. The developers of tools and IDEs should put into practice the expectations outlined in Section 5. However, it is not always obvious who should be responsible for what. Also, it is not always reasonable to expect IDE developers to respond to a feature request made by a plug-in developer. For instance, implementing IDE specific features such as enabling code navigation and search within the codebase should be the responsibility of the IDE developers. But implementing a new, hierarchical code visualization, for instance, is probably the tool-builder's job. To allow easier assignment of responsibility, we may consider the following questions. *"Can the plugin implement the expectation without the help of the IDE?" "Does the plugin fulfill its goal without the given expectation?" "Are the IDE and plugin necessary to tightly cooperate to implement the expectation?"*

5.3 Researchers perspective

Although SBFL has extensive literature, active work continues with new formula exploration, adding context to the spectrum, and developing different score calculation algorithms including AI-enabled approaches, to name a few. Researchers can take steps to prepare for different types of follow-up research.

Recommendation 5.3.1: Make data computed by SBFL tools easily available for researchers. One way to do this is to provide dedicated logging and dumping mechanisms to retrieve data from the tools. This can be useful for analyzing users' behavior as well.

Recommendation 5.3.2: Make SBFL tools' underlying logic customizable. Another way is to offer customization options within the tool itself. This can be achieved through settings, a modular architecture, or simply by making the tool open-source. While expecting researchers to include these customization systems out-of-the-box may be unrealistic, researchers should not prevent others from building upon their work either.

Recommendation 5.3.3: Easy implementation and experimentation with new algorithms. Devising new human-computer interactions for emerging scientific methods, like different flavors of SBFL, is presumably more suited for researchers than professional programmers. Hence, it is important for researchers to make it as easy as possible to implement new features and perform experimentation with prototype ideas.

Recommendation 5.3.4: Contribute to open science. Finally, we highly encourage tool developers to contribute to open science wherever they can. Besides the recommendations we already mentioned above, this would mean making the implementation, test data and other artifacts openly available. A possible framework that could be followed in particular is the *FAIR guideline for software* [20], which includes aspects like Findability, Accessibility, Interoperability and Reusability, among others.

6 THREATS TO VALIDITY

Our study focused on integrating SBFL tools into IDEs. However, we understand that there are several critical factors that could impact the robustness and applicability of our findings. We acknowledge

that the selection of specific tools and environments for this study was necessary, but it limits the breadth of our conclusions. Therefore, the insights gathered from this study may not be applicable to all varieties of these tools or different development environments. Future research could benefit from including a wider range of tools to broaden the scope of our findings.

It is important to recognize that the interpretation of feedback from users and developers is subjective and based on individual experiences and perceptions. Therefore, to minimize the impact of this subjectivity, future experiments should consider increasing the number of participants and implementing measures to ensure anonymity.

The methodology used to assess the efficacy and usability of tools integrated within IDEs may not encompass every facet of the user and developer experience, which could result in a partial evaluation. Therefore, a more comprehensive investigation that solicits feedback from a diverse group of users is necessary to offer a more holistic understanding of the subject matter.

Finally, the rapidly evolving nature of SBFL tools and IDEs poses a formidable challenge to the long-term relevance of research findings. It is crucial to periodically re-evaluate the efficacy of these tools, taking into account the latest advancements, to ensure that our research remains relevant and cutting-edge. Such periodic assessments are crucial for maintaining the credibility and applicability of our research outcomes.

7 CONCLUSIONS

This paper is a work towards practically usable SBFL in IDEs. Out of the total of 29 expectations, only 18 were implemented (62%), which implies that 11 expectations were left unfulfilled. It is noteworthy that only one of the unimplemented expectations was planned. Hence, it is safe to assume that the acceptance of SBFL tools by industrial Integrated Development Environment (IDE) users is still far from being achieved. The technique itself is promising since it could significantly help in semi-automated debugging, but currently the implementations are not mature enough to be considered by professional programmers.

Based on surveying existing tools, asking users' opinions, and our own experiences, we compiled a list of concrete recommendations for future tool builders for this task. It is without doubt that the underlying technology should also be further researched to reach the efficiency and precision that are required in a practical, everyday context as well, not only in a lab setting. However, our concerns regarding the implementation into IDEs should be equally considered, and parallel development of the two can hopefully give birth to usable SBFL plug-ins in some of the popular IDEs in the future.

ACKNOWLEDGEMENTS

This research was carried out in project TKP2021-NVA-09 supported by the Ministry of Innovation and Technology of Hungary from the National Research, Development and Innovation Fund, financed under the TKP2021-NVA funding scheme.

REFERENCES

- [1] Rui Abreu, Peter Zoetewij, and Arjan J.c. Van Gemund. 2006. An Evaluation of Similarity Coefficients for Software Fault Localization. In *2006 12th Pacific*

- Rim International Symposium on Dependable Computing (PRDC'06)*. 39–46. <https://doi.org/10.1109/PRDC.2006.18>
- [2] Rui Abreu, Peter Zoetewij, and Arjan J.C. Van Gemund. 2007. On the accuracy of spectrum-based fault localization. *Proceedings - Testing: Academic and Industrial Conference Practice and Research Techniques, TAIC PART-Mutation 2007* (2007), 89–98. <https://doi.org/10.1109/TAICPART.2007.4344104>
 - [3] Rui Abreu, Peter Zoetewij, and Arjan J.C. van Gemund. 2009. Spectrum-Based Multiple Fault Localization. In *2009 IEEE/ACM International Conference on Automated Software Engineering*. 88–99. <https://doi.org/10.1109/ASE.2009.25>
 - [4] Pragya Agarwal and Arun Prakash Agrawal. 2014. Fault-localization techniques for software systems: A Literature Review. *ACM SIGSOFT Software Engineering Notes* 39, 5 (sep 2014), 1–8. <https://doi.org/10.1145/2659118.2659125>
 - [5] Paul CH Albers and Han de Vries. 2001. Elo-rating as a tool in the sequential estimation of dominance strengths. *Animal Behaviour* (2001), 489–495.
 - [6] Archana and Ashutosh Agarwal. 2022. Evaluation of Spectrum Based Fault Localization Tools. In *15th Innovations in Software Engineering Conference (Gandhinagar, India) (ISEC 2022)*. Association for Computing Machinery, New York, NY, USA, Article 19, 5 pages. <https://doi.org/10.1145/3511430.3511470>
 - [7] José Campos, André Ribeiro, Alexandre Perez, and Rui Abreu. 2012. Gzoltar: An eclipse plug-in for testing and debugging. *2012 27th IEEE/ACM International Conference on Automated Software Engineering, ASE 2012 - Proceedings* (2012), 378–381.
 - [8] Cheng Chen and Nan Wang. 2017. UnitFL: A fault localization tool integrated with unit test. *Proceedings of 2016 5th International Conference on Computer Science and Network Technology, ICCSNT 2016* (2017), 136–142.
 - [9] Higor A. de Souza, Marcos L. Chaim, and Fabio Kon. 2016. Spectrum-based Software Fault Localization: A Survey of Techniques, Advances, and Challenges. (jul 2016), 1–46. arXiv:1607.04347 <http://arxiv.org/abs/1607.04347>
 - [10] Higor A. de Souza, Danilo Mutti, Marcos L. Chaim, and Fabio Kon. 2018. Contextualizing spectrum-based fault localization. *Information and Software Technology* 94 (2018), 245–261. <https://doi.org/10.1016/j.infsof.2017.10.014>
 - [11] Arpad E Elo. 1967. The proposed uscf rating system, its development, theory, and applications. *Chess Life* 22, 8 (1967), 242–247.
 - [12] Gao, Meng, Li, Pengyu, Chen, Congcong, and Jiang, Yunsong. 2018. Research on Software Multiple Fault Localization Method Based on Machine Learning. *MATEC Web Conf.* 232 (2018), 01060. <https://doi.org/10.1051/mateconf/201823201060>
 - [13] Liang Gong, David Lo, Lingxiao Jiang, and Hongyu Zhang. 2012. Interactive fault localization leveraging simple user feedback. In *2012 28th IEEE International Conference on Software Maintenance (ICSM)*. 67–76. <https://doi.org/10.1109/ICSM.2012.6405255>
 - [14] Dan Hao, Lu Zhang, Tao Xie, Hong Mei, and Jia Su Sun. 2009. Interactive Fault Localization Using Test Information. *Journal of Computer Science and Technology* 24, 5 (2009), 962–974. <https://doi.org/10.1007/s11390-009-9270-z>
 - [15] Ferenc Horváth, Árpád Beszédés, Béla Vancsics, Gergő Balogh, László Vidács, and Tibor Gyimóthy. 2022. Using contextual knowledge in interactive fault localization. *Empirical Software Engineering* 27, 6 (2022), 150.
 - [16] James A. Jones and Mary Jean Harrold. 2005. Empirical Evaluation of the Tarantula Automatic Fault-Localization Technique. In *Proceedings of the 20th IEEE/ACM International Conference on Automated Software Engineering* (Long Beach, CA, USA) (ASE '05). Association for Computing Machinery, New York, NY, USA, 273–282. <https://doi.org/10.1145/1101908.1101949>
 - [17] Fabian Keller, Lars Grunske, Simon Heiden, Antonio Filieri, Andre van Hoorn, and David Lo. 2017. A Critical Evaluation of Spectrum-Based Fault Localization Techniques on a Large-Scale Software System. In *2017 IEEE International Conference on Software Quality, Reliability and Security (QRS)*. 114–125. <https://doi.org/10.1109/QRS.2017.22>
 - [18] Andrew J. Ko and Brad A. Myers. 2008. Debugging reinvented: Asking and answering why and why not questions about program behavior. *Proceedings - International Conference on Software Engineering* (2008), 301–310.
 - [19] Pavneet Singh Kochhar, Xin Xia, David Lo, and Shanping Li. 2016. Practitioners' expectations on automated fault localization. In *Proceedings of the 25th International Symposium on Software Testing and Analysis - ISSTA 2016*. ACM Press, 165–176.
 - [20] Anna-Lena Lamprecht, Leyla Garcia, Mateusz Kuzak, Carlos Martinez, Ricardo Arcila, Eva Martin Del Pico, Victoria Dominguez Del Angel, Stephanie Van De Sandt, Jon Ison, Paula Andrea Martinez, Peter Mcquilton, Alfonso Valencia, Jennifer Harrow, Fotis Psomopoulos, Josep Ll. Gelpi, Neil Chue Hong, Carole Gobbe, Salvador Capella-gutierrez, Paul Groth, Paul Groth, and Michel Dumontier. 2020. Towards FAIR principles for research software. *Data Science* 3, 1 (12 June 2020), 37–59. <https://doi.org/10.3233/DS-190026>
 - [21] Cheng Li, Jindong Wang, Yixuan Zhang, Kaijie Zhu, Wenxin Hou, Jianxun Lian, Fang Luo, Qiang Yang, and Xing Xie. 2023. Large Language Models Understand and Can be Enhanced by Emotional Stimuli. arXiv:2307.11760 [cs.CL]
 - [22] Henrique Lemos Ribeiro, Higor Amario De Souza, Roberto Paulo Andrioli De Araujo, Marcos Lordello Chaim, and Fabio Kon. 2018. Jaguar: A Spectrum-Based Fault Localization Tool for Real-World Software. *Proceedings - 2018 IEEE 11th International Conference on Software Testing, Verification and Validation, ICST 2018* (2018), 404–409.
 - [23] Qusay Idrees Sarhan, Hassan Bapeer Hassan, and Árpád Beszédés. 2023. SFLaaS: Software Fault Localization as a Service. In *2023 IEEE Conference on Software Testing, Verification and Validation (ICST)*. 467–469. <https://doi.org/10.1109/ICST57152.2023.00052>
 - [24] Qusay Idrees Sarhan and Árpád Beszédés. 2022. A Survey of Challenges in Spectrum Based Software Fault Localization. *IEEE Access* 10 (2022), 10618–10639. <https://doi.org/10.1109/ACCESS.2022.3144079>
 - [25] Qusay Idrees Sarhan, Attila Szatmári, Rajmond Tóth, and Árpád Beszédés. 2021. CharmFL: A Fault Localization Tool for Python. In *2021 IEEE 21st International Working Conference on Source Code Analysis and Manipulation (SCAM)*. 114–119. <https://doi.org/10.1109/SCAM52516.2021.00022>
 - [26] Attila Szatmári, Qusay Idrees Sarhan, Gergő Balogh, Péter Attila Soha, and Árpád Beszédés. 2023. *IDE Workshop 2024 Online Appendix*. https://www.inf.u-szeged.hu/~szatma/IDE2024/online_appendix/ Accessed: December 7, 2023.
 - [27] Attila Szatmári, Qusay Idrees Sarhan, and Árpád Beszédés. 2022. Interactive Fault Localization for Python with CharmFL. In *Proceedings of the 13th International Workshop on Automating Test Case Design, Selection and Evaluation* (Singapore, Singapore) (A-TEST 2022). Association for Computing Machinery, New York, NY, USA, 33–36. <https://doi.org/10.1145/3548659.3561312>
 - [28] Nan Wang, Zheng Zheng, Zhenyu Zhang, and Cheng Chen. 2015. FLAVS: A fault localization add-in for visual studio. *Proceedings - 1st International Workshop on Complex Faults and Failures in Large Software Systems, COUFLESS 2015* (2015), 1–6.
 - [29] Wanzhi Wen. 2012. Software fault localization based on program slicing spectrum. In *2012 34th International Conference on Software Engineering (ICSE)*. 1511–1514. <https://doi.org/10.1109/ICSE.2012.6227049>
 - [30] W. Eric Wong, Ruizhi Gao, Yihao Li, Rui Abreu, and Franz Wotawa. 2016. A Survey on Software Fault Localization. *IEEE Transactions on Software Engineering* 42, 8 (aug 2016), 707–740. <https://doi.org/10.1109/TSE.2016.2521368>